



Estudiantes: Camilo Andres Lievano Rendon, Juan Mateo Roza Torres, Juan Diego Lasso Montaña, Rene Alejandro Barrero Gonzalez.

Lenguaje de programación y framework seleccionado 

Para el desarrollo del sistema ArchiBox se ha seleccionado el lenguaje Python junto con el framework PyQt para la creación de la interfaz gráfica. Esta combinación resulta especialmente adecuada para la implementación de aplicaciones de escritorio, sin necesidad de depender de navegadores web o servidores externos.

Comparativo de tecnologías consideradas:

Tecnología	Ventajas 	Desventajas 
Python + PyQt	-Sintaxis simple y rápida de desarrollar. -Interfaz gráfica profesional y multiplataforma. -Abundante documentación. -Compatible con SQLite y bibliotecas de seguridad como bcrypt. -Ideal para ejecución en red local sin configuración de servidores.	-Ejecutables grandes si se empaqueta como .exe. -Requiere PyInstaller u otra herramienta para distribución. -No tan óptimo como C++ en rendimiento gráfico puro.
Java + JavaFX	-Muy estable y potente para escritorio. -Buena separación entre lógica y vista (FXML).	-Curva de aprendizaje mayor. -Mayor peso y complejidad en el despliegue. -Verbosidad del lenguaje.
Electron.js	-Interfaz moderna con tecnologías web (HTML, CSS, JS). -Experiencia de usuario tipo navegador embebido.	-Consumo elevado de recursos para una aplicación simple. -Genera ejecutables grandes (~200 MB o más). -Requiere conocimientos avanzados de frontend/backend.

Aplicaciones web con Flask/React	-Muy escalables. -Desplegables fácilmente en la nube o intranet.	-Requiere servidor local o navegador web. -Complica la instalación en entornos sin experiencia técnica. -Mayor fragmentación del stack (frontend + backend).
---	---	--

Base de datos relacional seleccionada 

Se utilizará SQLite como base de datos relacional. SQLite es ampliamente reconocido por su simplicidad, confiabilidad y ligereza. Su funcionamiento no requiere un servidor de bases de datos aparte, ya que guarda la información en un solo archivo .db en disco, lo que facilita su uso en aplicaciones monolíticas y distribuciones de escritorio.

Justificación de uso :

- ❖ No necesita instalación ni configuración de un gestor como PostgreSQL o MySQL.
- ❖ Es suficiente para el volumen estimado del sistema (hasta 5.000 libros y 50 usuarios activos).
- ❖ Compatible con Python sin requerir adaptadores adicionales.
- ❖ Ideal para entornos controlados y de red local.
- ❖ Soporta integridad referencial, índices, claves primarias y consultas SQL estándar.

Bibliotecas y herramientas complementarias 

Para complementar el desarrollo del sistema, se utilizarán las siguientes bibliotecas y herramientas:

- ❖ Qt Designer : para la creación visual de formularios .ui, que serán integrados en PyQt.
- ❖ SQLite3 : para la conexión con la base de datos.
- ❖ SQLAlchemy : Para la interacción con la base de datos.
- ❖ bcrypt : para el cifrado seguro de contraseñas, cumpliendo con los requerimientos de seguridad.

- ❖ PyInstaller : para generar ejecutables .exe para Windows.
- ❖ datetime y logging : para el registro de actividades y trazabilidad del sistema (auditoría).
- ❖ reportlab o fpdf : para generar reportes en PDF desde los datos del sistema.
- ❖ os, shutil, pathlib : para gestionar archivos digitalizados, nombres, rutas y almacenamiento local.

Justificación de las elecciones :

1. **Arquitectura monolítica y red local** : Python + PyQt + SQLite permiten un despliegue sencillo en red local sin necesidad de conexión a internet, servidores web, ni configuraciones complejas. Esto facilita su uso en situaciones donde los recursos técnicos pueden ser limitados.
2. **Enfoque en usabilidad para personal no técnico** : PyQt permite construir interfaces gráficas modernas y funcionales que se sienten como aplicaciones nativas, facilitando su adopción.
3. **Desarrollo rápido y mantenible** : Python reduce considerablemente la carga de desarrollo comparado con lenguajes más verbosos como Java. Su disponibilidad de bibliotecas facilita el cumplimiento de requisitos sin reinventar funcionalidades.
4. **Seguridad y trazabilidad** : El uso de bibliotecas como bcrypt para el manejo seguro de contraseñas, y logging para auditoría de acciones, permiten cumplir con los requisitos legales y de integridad de la información.
5. **Portabilidad y distribución** : Gracias a herramientas como PyInstaller, se puede generar un instalador ejecutable sin dependencias externas.
6. **Adecuación a la escala del sistema** : SQLite es suficiente para el tamaño proyectado y permite manejar la base de datos de forma eficiente, sin necesidad de una infraestructura de servidor adicional.